

Implement Client Script & UI Policy (Incident)

Phase 7 – Project Documentation

7.1 Purpose

The purpose of this phase is to prepare complete technical and user documentation for the **Implement Client Script & UI Policy (Incident)** project.

The documentation records the project problem, user story, objectives, requirements, system design, ServiceNow implementation, testing activities, demonstration process, and final outcome.

7.2 Project Title

Implement Client Script & UI Policy (Incident)

The project is implemented using the **ServiceNow Incident Management** application.

7.3 Problem Statement

Incident forms may require dynamic field control and validation to ensure that users enter appropriate information and follow the required process.

Without suitable configuration, important fields may be left incomplete, values may not be updated automatically, and users may make unwanted changes directly from Incident list views.

The project addresses these requirements by implementing **UI Policies and Client Scripts** to control Incident form behaviour and improve data consistency.

7.4 User Story

As a ServiceNow user, I want the Incident form to automatically control fields based on the Incident Impact and validate important information, so that Incidents can be submitted correctly and unwanted list-based changes can be prevented.

7.5 Project Objectives

The main objectives of the project are:

- To implement a UI Policy for High Impact Incidents.

- To make **Assignment Group** mandatory when Impact is High.
 - To control the **Urgency** field dynamically.
 - To automatically set Urgency to High when Impact is High.
 - To prevent submission when Assigned To is missing for a High Impact Incident.
 - To prevent direct State changes through Incident list editing.
 - To allow valid State changes through the Incident form.
 - To test the implemented configurations under different conditions.
-

7.6 Requirements

7.6.1 Functional Requirements

The project requires the following functionality:

ID	Requirement
FR-01	The system must provide an Incident form.
FR-02	A UI Policy must be applied when Impact is High.
FR-03	Assignment Group must become mandatory when Impact is High.
FR-04	Urgency must become read-only when Impact is High.
FR-05	Urgency must automatically be set to High when Impact changes to High.
FR-06	A High Impact Incident must not be submitted when Assigned To is empty.
FR-07	Direct State changes through Incident list editing must be prevented.
FR-08	State changes through the Incident form must be allowed.

7.6.2 Non-Functional Requirements

- The configuration should be simple to use.
 - The implemented scripts should respond correctly to user actions.
 - The configuration should maintain consistent Incident data.
 - The implementation should be testable within the ServiceNow Incident Management environment.
-

7.7 System Design

The implemented project follows the following logical flow:

Incident Form → Impact = High → UI Policy / Client Scripts → Field Control & Validation → Incident Update

The main components are:

- **High Impact Control** – UI Policy
 - **Urgency** – UI Policy Action
 - **Auto set urgency for high impact** – onChange Client Script
 - **Prevent save if Assigned To missing** – onSubmit Client Script
 - **Prevent state change via list edit** – onCellEdit Client Script
-

7.8 Implementation

7.8.1 UI Policy

The project uses a UI Policy named **High Impact Control**.

Configuration:

- **Table:** Incident
- **Condition:** Impact is 1 – High
- **Active:** Yes
- **Assignment Group:** Mandatory
- **Reverse if false:** Yes

When Impact is High, Assignment Group becomes mandatory.

7.8.2 UI Policy Action

The UI Policy Action controls the **Urgency** field.

Configuration:

- **Field:** Urgency
- **Read-only:** Yes
- **Visible:** Unchanged

When Impact is High, Urgency becomes read-only.

7.8.3 onChange Client Script

Name: Auto set urgency for high impact

The script runs when the Impact field changes. When Impact is set to High, Urgency is automatically set to High and an informational message is displayed.

7.8.4 onSubmit Client Script

Name: Prevent save if Assigned To missing

The script checks whether Impact is High and Assigned To is empty. If both conditions are satisfied, the Incident submission is prevented and an error message is displayed.

7.8.5 onCellEdit Client Script

Name: Prevent state change via list edit

The script runs when the State field is edited directly from the Incident list. It displays a warning and prevents the list-based State change.

7.9 Testing

The implemented project was tested using the following activities:

7.9.1 Mandatory Enforcement Test

A High Impact Incident was tested without an Assigned To value.

Expected Result: The Incident should not be saved and an error message should be displayed.

7.9.2 Successful Save Test

A High Impact Incident was tested after assigning a valid user.

Expected Result: The Incident should save successfully.

7.9.3 Reverse Condition Test

Impact was changed from High to Medium.

Expected Result: The UI Policy behaviour should be reversed and the applicable fields should return to their normal state.

7.9.4 List Edit Blocking Test

The State field was edited directly from the Incident list.

Expected Result: The onCellEdit Client Script should display a warning and prevent the State change.

7.9.5 Form-Based Update Test

The State field was changed through the Incident form.

Expected Result: The State change should be saved successfully.

7.10 Demonstration

The project can be demonstrated using the following sequence:

1. Open the **Incident** module.
2. Open or create an Incident.
3. Set **Impact** to **1 – High**.
4. Show that **Assignment Group** becomes mandatory.
5. Show that **Urgency** becomes read-only.
6. Change Impact and demonstrate the automatic Urgency behaviour.
7. Test submission with Assigned To empty.
8. Assign a valid user and submit the Incident.
9. Open the Incident list.
10. Attempt to edit State directly from the list.
11. Show that the onCellEdit Client Script blocks the change.
12. Open the Incident form and demonstrate a form-based State update.

7.11 Evidence and Screenshots

Project evidence is supported by screenshots captured from the ServiceNow environment.

The evidence includes:

7.11.1 High Impact Control UI Policy

servicenow

All
Favorites
History
Workspaces
Admin

UI Policy - High Impact Control

Search

UI Policy

High Impact Control

View: Advanced

Update

Delete

UI policies change fields on a form based on a set of conditions. Use UI policies to show or hide fields, or to make fields mandatory or read-only based on these conditions. A UI policy specifies one or more actions to take when the policy is triggered. First, create the policy. Then add as many actions as necessary. UI policy actions are applied only if all of the following conditions are met:

- The UI Policy is **Active**
- The items in the **Conditions** field evaluate to true
- The field specified in the UI policy action is present on the specified form

[More info](#)

Table

Incident [incident]

Application

Global

Active

☒

* Short description

High Impact Control

Order

100

When to Apply

Script

Conditions

Add Filter Condition

Add OR Clause

Impact

is

1 - High

AND

OR

X

If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific

Apply the UI policy actions when the form is loaded and when the user changes values on the form

Global

☒

On load

☒

Reverse the effects of the UI policy actions when the Conditions evaluate to false

Tables that extend the specified Table inherit this UI Policy

Reverse if false

☒

Inherit

☐

7.11.2 Urgency UI Policy Action

servicenow

All
Favorites
History
Admin

UI Policy Action - urgency

Search

UI Policy Action

urgency

Update

Delete

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

UI policy

High Impact Control

Table

Incident [incident]

* Field name

Urgency

Application

Global

Mandatory

Leave alone

Visible

Leave alone

Read only

True

Clear the field value

☐

Update

Delete

7.11.3 onChange Client Script

The screenshot shows the ServiceNow Client Script configuration interface. The title bar indicates the script is named 'Auto set urgency for high impact'. A blue informational banner at the top states: 'New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.'

The configuration fields are as follows:

- Name: Auto set urgency for high impact
- Table: Incident [Incident]
- UI Type: Desktop
- Type: onChange
- Field name: Impact
- Application: Global
- Active: ☒
- Inherited: ☐
- Global: ☒

Description and Messages fields are empty.

The Script editor contains the following code:

```
1 function onChange(control, oldValue, newValue, isLoading) {  
2   if (isLoading || newValue == '') {  
3     return;  
4   }  
5  
6   if (newValue == '1') {  
7     g_form.setValue('urgency', '1');  
8     g_form.addInfoMessage('Urgency set to High for High impact incident.');9   }  
10 }  
11
```

The 'Isolate script' checkbox is checked.

7.11.4 onSubmit Client Script

The screenshot shows the ServiceNow Client Script configuration interface. The title bar indicates the script is named 'Prevent save if Assigned To missing'. A blue informational banner at the top states: 'New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.'

The configuration fields are as follows:

- Name: Prevent save if Assigned To missing
- Table: Incident [Incident]
- UI Type: Desktop
- Type: onSubmit
- Application: Global
- Active: ☒
- Inherited: ☐
- Global: ☒

Description and Messages fields are empty.

The Script editor contains the following code:

```
1 function onSubmit() {  
2   if (g_form.getValue('impact') == '1' &&  
3     g_form.getValue('assigned_to') == '') {  
4  
5     g_form.showErrorMessage(  
6       'assigned_to',  
7       'Assigned To is mandatory for High impact incidents.'  
8     );  
9     return false;  
10  }  
11  return true;  
12 }
```

The 'Isolate script' checkbox is checked.

7.11.5 onCellEdit Client Script

The screenshot shows the ServiceNow interface for configuring a Client Script. The title bar indicates 'Client Script - Prevent state change via list edit'. A blue information banner at the top states: 'New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client_globals" to false.'

The configuration form includes the following fields:

- Name:** Prevent state change via list edit
- Table:** Incident [Incident]
- UI Type:** Desktop
- Type:** onCellEdit
- Field name:** State
- Application:** Global
- Active:** ☒
- Inherited:** ☐
- Global:** ☒
- Description:** (empty text area)
- Messages:** (empty text area)
- Script:**

```
1 function onCellEdit(sysIDs, table, oldvalues, newvalue, callback) {  
2  
3     alert('State cannot be updated using list editing. Please open the Incident.');4  
5     callback(false);  
6 }
```
- Isolate script:** ☒

At the bottom, there are 'Update' and 'Delete' buttons.

7.11.6 Mandatory enforcement test

The screenshot shows the ServiceNow 'Incident - Create INC0010005' form. A blue banner at the top indicates 'Urgency set to High for High impact incident.' Below this, a red error banner states: 'The following mandatory fields are not filled in: Short description, Assignment group'.

The form fields are organized into two columns:

- Left Column:**
 - Number: INC0010005
 - * Caller: Abel Tuter
 - Category: Inquiry / Help
 - Subcategory: -- None --
 - Service: (empty)
 - Service offering: (empty)
 - Configuration item: (empty)
 - * Short description: Hii
- Right Column:**
 - Channel: -- None --
 - State: New
 - Impact: 1 - High
 - Urgency: 1 - High
 - Priority: 1 - Critical
 - * Assignment group: CAB Approval
 - Assigned to: (empty)

Two red error messages are displayed at the bottom right:

- Assigned To is mandatory for High impact incidents.
- Assigned To is mandatory for High impact incidents.

At the top right, there are 'Submit' and 'Resolve' buttons.

7.11.7 Successful save test

The screenshot shows the ServiceNow Incident form for INC0010005. The form is in 'In Progress' state with a '1 - Critical' priority. The caller is 'Abel Tuter' and the short description is 'Hii'. The form includes fields for Number, Channel, State, Impact, Urgency, Priority, Assignment group, Assigned to, Service, Service offering, Configuration item, and Description. The 'Related Search Results' section is visible below the form fields.

Number: INC0010005

* Caller: Abel Tuter

Category: Inquiry / Help

Subcategory: -- None --

Service:

Service offering:

Configuration item:

* Short description: Hii

Description:

Channel: -- None --

State: In Progress

Impact: 1 - High

Urgency: 1 - High

Priority: 1 - Critical

* Assignment group: CAB Approval

Assigned to: Bernard Laboy

Related Search Results >

Notes | Related Records | Resolution Information

Watch list | Work notes list

7.11.8 Reverse condition test

The screenshot shows the ServiceNow Incident form for INC0010005. The form is in 'In Progress' state with a '2 - High' priority. The caller is 'Abel Tuter' and the short description is 'Hii'. The form includes fields for Number, Channel, State, Impact, Urgency, Priority, Assignment group, Assigned to, Service, Service offering, Configuration item, and Description. The 'Related Search Results' section is visible below the form fields.

Number: INC0010005

* Caller: Abel Tuter

Category: Inquiry / Help

Subcategory: -- None --

Service:

Service offering:

Configuration item:

* Short description: Hii

Description:

Channel: -- None --

State: In Progress

Impact: 2 - Medium

Urgency: 1 - High

Priority: 2 - High

Assignment group: CAB Approval

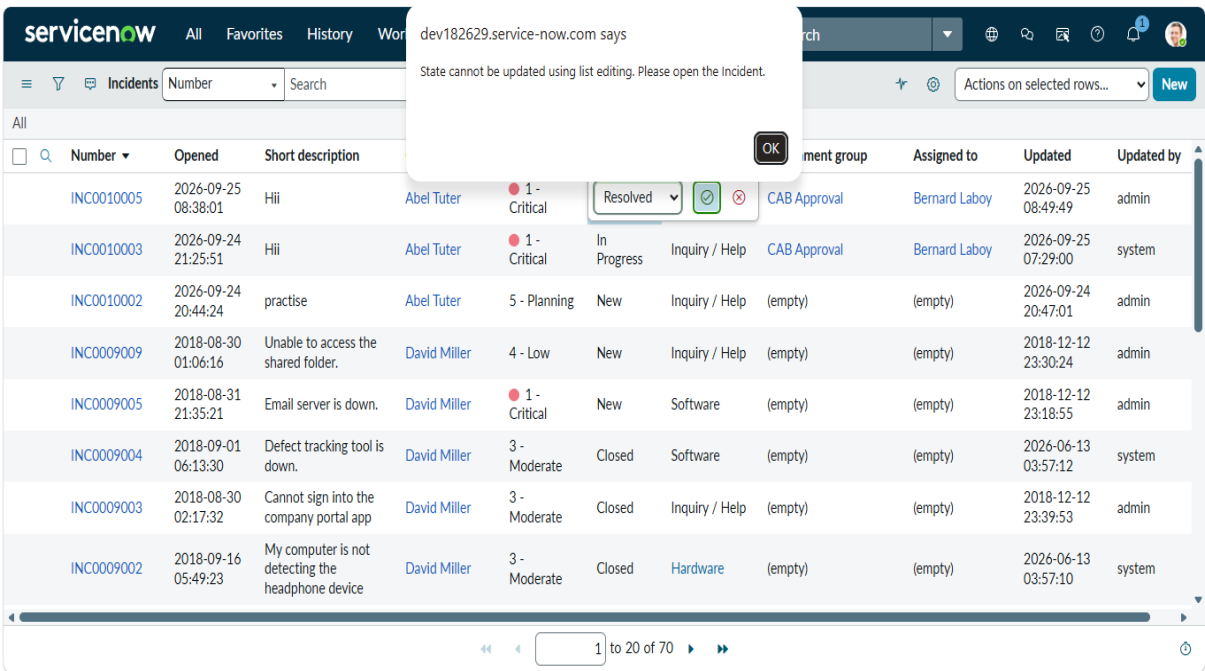
Assigned to: Bernard Laboy

Related Search Results >

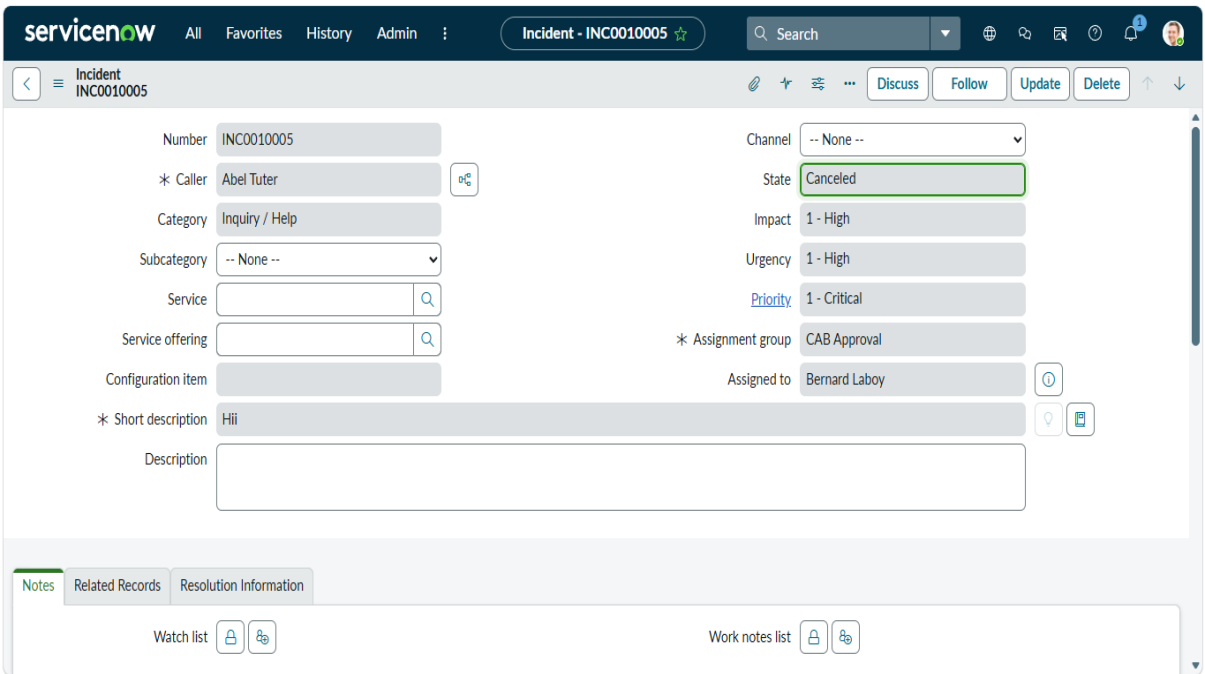
* Notes | Related Records | Resolution Information

Watch list | Work notes list

7.11.9 State list-edit blocking test



7.11.10 Form-based State update test



These screenshots provide evidence of the implemented configuration and testing activities.

7.12 Conclusion

The **Implement Client Script & UI Policy (Incident)** project demonstrates the practical use of ServiceNow UI Policies and Client Scripts for controlling Incident form behaviour.

The implementation provides dynamic field control, automatic value updates, submission validation, and restrictions on direct list editing. The configured components were tested under different Incident conditions to verify their intended behaviour.